

As a first step, write the following code to link to the ML5.js library from the browser, and to print the ML5 version in the console log window.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Getting Started with ml5.js</title>
  <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
  <script src="https://unpkg.com/
ml5@0.4.3/dist/ml5.min.js"></script>
</head>

<body>
  <script>
    console.log('ml5 version:', ml5.
version);
  </script>
</body>
</html>
```

The CDN link to ML5.js is provided as shown below:

```
<script src="https://unpkg.com/
ml5@0.4.3/dist/ml5.min.js"></script>
```

If you open the aforementioned code in a browser and look at the console log, you will get the version that will look something similar to what's given below:

```
ml5 version: 0.4.3
```

ML5.js has support for the following categories:

- Helpers
- Image
- Sound

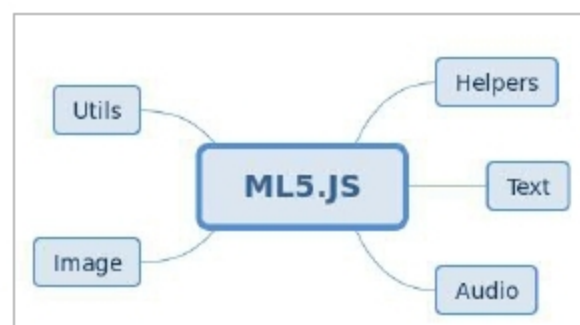


Figure 1: ML5.js components

- Text
- Utils

The image, sound and text are used to apply data belonging to the respective type. The *helpers* category includes components such as NeuralNetworks and FeatureExtractor, which can be used to perform transfer learning.

An example of a simple neural network with ML5.js

Try the following code to get the basic neural network functionality in ML5.js:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Getting Started with ml5.
js</title>
  <meta name="viewport"
content="width=device-width, initial-
scale=1.0">
  <script src="https://unpkg.com/
ml5@0.4.3/dist/ml5.min.js"></script>
</head>

<body>
  <script>

// Initialize the the neural network
const neuralNetwork = ml5.
neuralNetwork(1, 1);

// add in some data
for(let i = 0; i < 100; i+=1){
  const x = i;
  const y = i * 2;
  neuralNetwork.data.addData( [x],
[y])
}

// normalize your data
neuralNetwork.data.normalize();
// train your model
neuralNetwork.train(finishedTraining);

// when it is done training, run
.predict()
function finishedTraining(){
  neuralNetwork.predict( [60], (err,
results) => {
```

```
    console.log(results);
  })
}
</script>
</body>
</html>
```

If you execute this code in the browser, you will get the following output:

```
{value: 120.65793764591217, label:
"output0"}
```

You may have observed that the output value is input multiplied by 2. As the input here is 60, the output value is 120.65793.

Using a pre-trained model

The FeatureExtractor component can be used to extract the already learned features from the existing models such as MobileNet.

A simple example to load the MobileNet and classify the images is shown below:

```
// Extract the already learned features
from MobileNet
const featureExtractor = ml5.
featureExtractor("MobileNet",
modelLoaded);

// When the model is loaded
function modelLoaded() {
  console.log("Model Loaded!");
}

// Create a new classifier using those
features and with a video element
const classifier = featureExtractor.
classification(video, videoReady);

// Triggers when the video is ready
function videoReady() {
  console.log("The video is ready!");
}

// Add a new image with a label
classifier.addImage(document.
getElementById("dogA"), "dog");
```